

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	R KALAI AMUDHAN	Reg. No.:	192511292
Date:		Duration:	60 minutes

Code of Conduct

- I R KALAI AMUDHAN (192511292) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
- Signature of the Student: R Kalai Amudhan

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, design a CI/CD (Continuous Integration and Continuous Deployment) pipeline for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. **Identify suitable tools and technologies** for each stage of the pipeline and justify their selection.
4. **Define appropriate automated testing strategies** to be integrated into the pipeline.
5. **Design the deployment strategy**, including development, testing/staging, and production environments.
6. **Incorporate appropriate security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. **Prepare a CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a CI/CD Pipeline Design document containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis (15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow (25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection (20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration (15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy (15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis (15%)	/5
Pipeline Architecture & Workflow (25%)	/4
Tools & Technology Selection (20%)	/4
Automated Testing & Quality Integration (15%)	/4
Deployment, Security & Rollback Strategy (15%)	/4
Pipeline Diagram & Documentation (10%)	/4
TOTAL	/25

ANNEXURE B – COMPLETED CI/CD PIPELINE DESIGN

Problem Statement 17: Intelligent AI-Based Smart Dairy Farm Management System

The Intelligent AI-Based Smart Dairy Farm Management System combines artificial intelligence, IoT wearable sensors, milk-production tracking, feeding optimization, cloud services, and real-time analytics into a single operational platform. Because the platform directly influences animal welfare and farm income, software changes cannot be released casually; every change must be integrated, automatically verified, security-checked, and deployed through a controlled, repeatable process. This document presents a complete CI/CD (Continuous Integration and Continuous Deployment) design for the system, covering requirement analysis, pipeline architecture, tool selection, automated testing, security, deployment strategy, monitoring, and rollback, together with supporting diagrams for the assigned Problem Statement 17.

1. Requirement & Pipeline Analysis

A reliable CI/CD process is required so that frequent software changes can be integrated, tested, secured, and deployed without unnecessary disruption to farm operations. The pipeline requirements and the underlying system requirements are analysed below.

1.1 CI/CD Process Requirements

- Use Git-based source control with protected branches and pull requests.
- Automatically build every valid code change and detect compilation or dependency errors early.
- Test sensor ingestion, milk-production calculations, feeding logic, APIs, dashboards, and AI services.
- Validate incoming farm data for missing, invalid, duplicated, or abnormal values.
- Perform code-quality, dependency, secret, and container security checks before release.
- Deploy first to a staging environment that closely represents production.
- Use production health checks, monitoring, notifications, and rollback to protect live services.

1.2 Functional Requirements of the Smart Dairy System

The system must collect and process information related to cattle health, milk production, feeding schedules, and farm environmental conditions. The CI/CD pipeline must therefore validate the services responsible for these functions before they are released.

- The platform should register cattle and maintain identifiable records for each animal.
- Wearable and environmental sensors should provide regular readings to the software platform.
- The system should calculate and display milk-production information for individual cattle and the overall farm.
- The feeding component should support schedules and recommendations based on available farm information.
- The AI service should analyse selected health indicators and generate a disease-risk or health-risk result.
- The dashboard should present farm analytics and alerts in a form that can be understood by farm operators.
- Reports should be generated from validated data so that management decisions are based on reliable information.

1.3 Non-Functional Requirements

- Reliability: critical services should be continuously monitored and failures should be detected quickly.
- Security: application credentials, sensor data, user accounts, and cloud resources should be protected.
- Scalability: the architecture should allow more cattle, sensors, users, and farms to be added.
- Maintainability: source code should be version-controlled, tested, documented, and reviewed.
- Performance: dashboards and data-processing services should respond within acceptable operational limits.
- Traceability: every production version should be traceable to its source commit and build.

2. Pipeline Architecture & Workflow

The workflow starts when a developer commits code or raises a pull request. The CI system retrieves the repository, builds the application, runs automated tests, performs quality and security analysis, and creates a versioned deployable artifact. A successful build is deployed to staging for validation. After approval and successful health checks, the same validated artifact is deployed to production. Monitoring continues after release, and the previous stable version can be restored if a deployment fails.

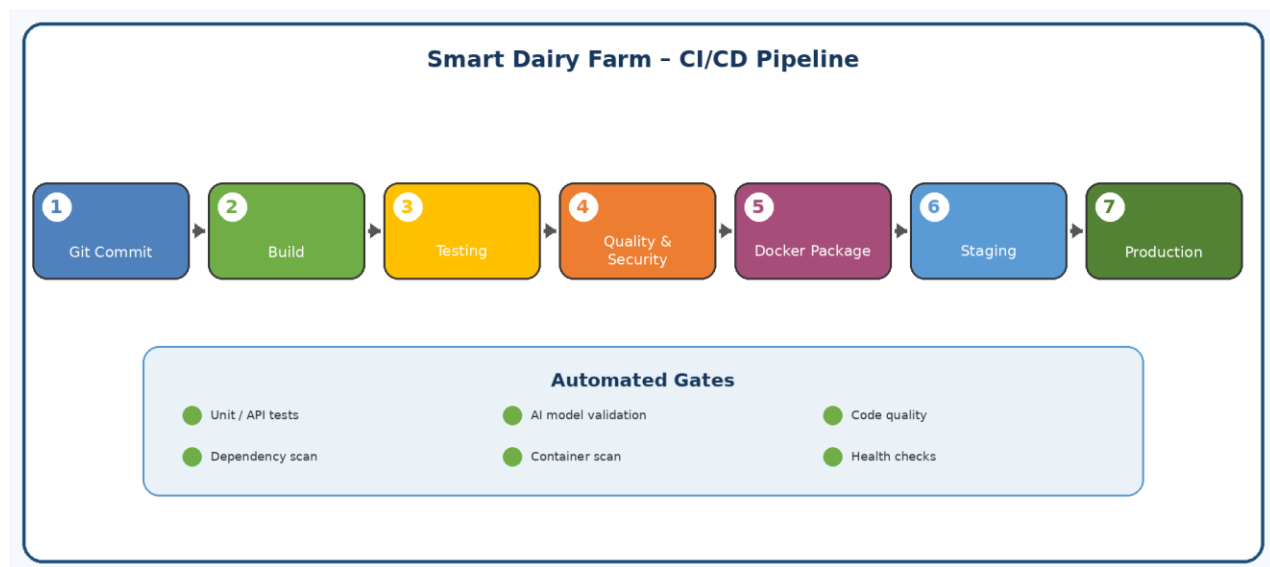


Figure 1. Colour CI/CD pipeline showing automated stages and release gates for the Smart Dairy Farm Management System

2.1 Stage-by-Stage Explanation

- **Developer Commit / Pull Request** – a developer commits application or infrastructure changes to the Git repository; the change is reviewed before it is merged into the protected main branch.
- **Build** – the CI runner installs dependencies and builds the application, services, and supporting components.
- **Automated Testing** – unit, integration, API, data-validation, and selected AI-model tests are executed.
- **Quality and Security** – static analysis, dependency scanning, secret detection, and container scanning are performed.
- **Package** – the validated application is converted into a versioned deployment artifact or Docker image and stored in a registry.
- **Staging** – the release candidate is deployed and checked in a production-like environment.

3. Tools and Technology Selection

The tools below were selected because they integrate naturally with one another, are widely supported, and cover every pipeline stage from source control to production monitoring.

Category	Tool / Technology	Purpose	Reason for Selection
Source Control	Git + GitHub	Manage source and reviews	Reliable version history and pull requests
CI/CD	GitHub Actions	Automate pipeline stages	Integrates directly with repository workflows
Containers	Docker	Package services	Consistent runtime across environments
Testing	PyTest / Jest / Postman	Automated validation	Covers application and API behaviour
Quality	SonarQube	Code-quality checks	Identifies maintainability issues
Security	Trivy	Container and dependency scan	Finds known vulnerabilities
Registry	Container Registry	Store images	Versioned and controlled artifacts
Cloud	AWS / Azure	Host services	Scalable cloud infrastructure
Monitoring	Prometheus + Grafana	Metrics and dashboards	Supports operational visibility

3.1 Why Tool Integration Matters

Using connected tools allows one pipeline to carry a change from source control through validation and deployment. The objective is not simply to use many tools, but to create a repeatable chain in which the output of one stage becomes the controlled input of the next stage.

4. Automated Testing and Quality Integration

Automated testing is integrated directly into the CI pipeline so that defects are detected before software reaches the production environment. Testing covers application logic, service communication, farm data processing, and AI-related components.

4.1 Testing Levels

- **Unit Testing** – verifies individual functions such as milk calculations, feeding rules, alert conditions, and data validation.
- **Integration Testing** – checks communication between the API, database, sensor ingestion service, AI service, and dashboard.
- **API Testing** – verifies request validation, response structure, authentication behaviour, and error handling.
- **Data Testing** – checks missing values, invalid readings, duplicate events, timestamps, and abnormal sensor values.

-
- **AI Model Testing** – evaluates the model using controlled validation data and checks whether performance remains within the accepted range.
 - **UI Testing** – verifies important dashboard functions such as login, cattle records, milk reports, and alerts.
 - **Smoke Testing** – confirms that the most important services work immediately after deployment.
 - **Regression Testing** – ensures that a new change has not broken previously working functionality.

4.2 Quality Gates

Quality gates are placed before the packaging and production stages. A failed critical test, severe security vulnerability, or unacceptable quality result stops the pipeline. This prevents an apparently successful build from becoming an unsafe production release.

4.3 Test Reporting

The CI system retains test results and makes failures visible to developers. Reports are attached to pipeline runs so that a failed deployment can be traced back to the exact stage and test case that caused the failure.

5. Security, Quality Assurance and Compliance

5.1 Application Security

- Use secure authentication and authorization for dashboard and administrative functions.
- Protect credentials and API keys using environment secrets or a dedicated secrets manager.
- Use encrypted communication (HTTPS/TLS) for network traffic where applicable.
- Apply least-privilege permissions to services and CI/CD identities.
- Review third-party dependencies and remove unnecessary packages.
- Scan container images before they are promoted to production.

5.2 CI/CD Security Controls

- Protect the main branch from direct, unreviewed changes.
- Require pull-request review for important changes.
- Restrict deployment permissions to approved workflows.
- Keep CI/CD secrets outside source code.
- Record deployment history and audit events.
- Use signed or otherwise traceable release artifacts where supported.

5.3 Quality Assurance

Quality assurance is continuous throughout the pipeline. Developers receive fast feedback from automated checks, while staging provides an additional opportunity to verify the complete system before production. This approach supports reliable releases without depending entirely on manual inspection.

6. Deployment Environments and Release Strategy

6.1 Development Environment

The development environment is used for implementation, debugging, local testing, and early integration. Developers can work on branches without affecting the production system.

6.2 Staging Environment

The staging environment receives a release candidate after the CI checks pass. It uses production-like services and configurations so that integration problems can be identified before release.

6.3 Production Environment

Production is the live environment used by farm operators. Only validated and approved artifacts are deployed. Monitoring and health checks are active immediately after deployment.

6.4 Deployment Approaches

- **Rolling deployment** – updates services gradually while maintaining availability.
- **Blue-green deployment** – maintains two environments and switches traffic after validation.
- **Canary deployment** – releases a new version to a small portion of users or services before wider rollout.

The selected method depends on the system's infrastructure, availability requirements, and operational complexity.

6.5 Rollback Strategy

Every production deployment retains the previous stable version. If health checks, error rates, or critical functions indicate a failed release, the deployment process restores the previous version and notifies the responsible team. Logs and audit records are maintained for troubleshooting and operational accountability.

7. Smart Dairy Farm System Architecture

The CI/CD process supports a multi-layer smart dairy platform. Wearable and environmental sensors provide farm information, application services process the data, AI components generate predictions, cloud services store and analyse information, and the dashboard presents results to farm operators. Organising the system into layers also allows the development team to test and deploy individual services while maintaining a clear overall architecture.

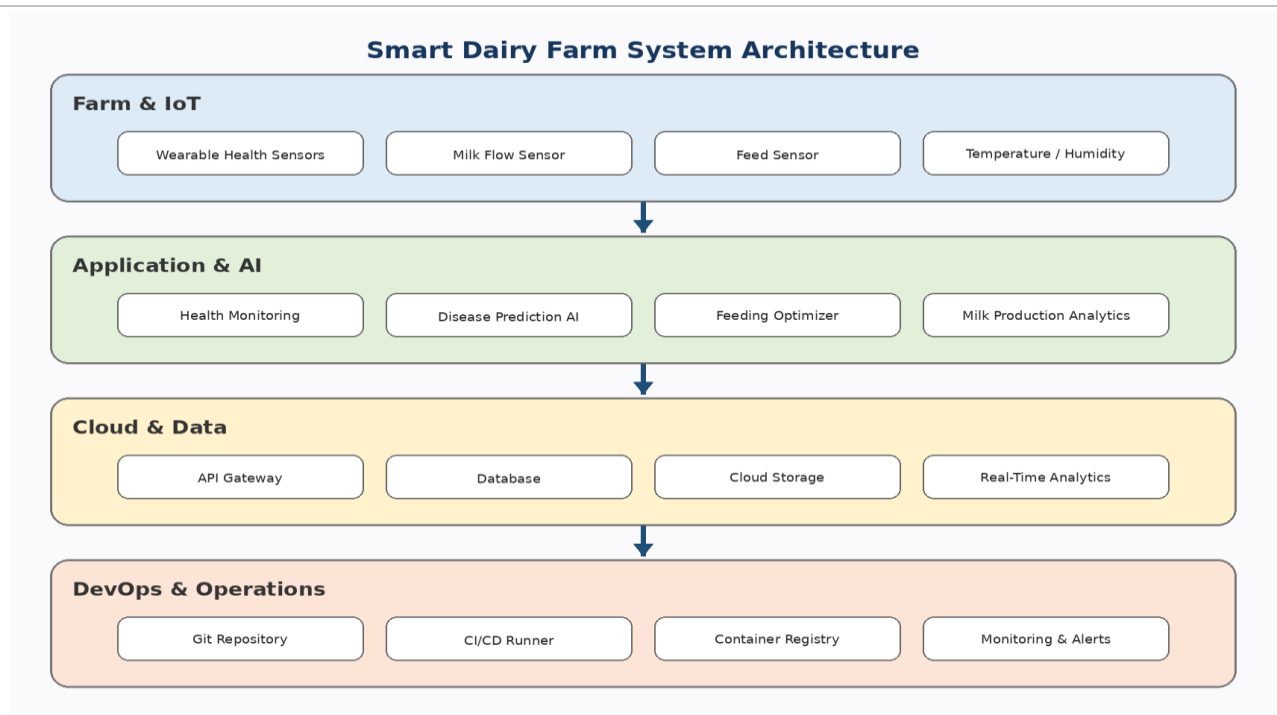


Figure 2. Colour high-level architecture of the Smart Dairy Farm Management System

8. Farm Monitoring, Data Flow and Analytics

The deployed platform receives cattle-health, feeding, milk-production, and environmental information. Sensor readings are collected from the farm and passed through an IoT/gateway layer. Application services validate and process the information; the cloud/data layer stores records and provides analytics, while AI services use appropriate data to generate predictions. The dashboard then presents the resulting information to the farmer or authorized user, supporting faster operational decisions while allowing software components to be updated through the controlled CI/CD workflow.

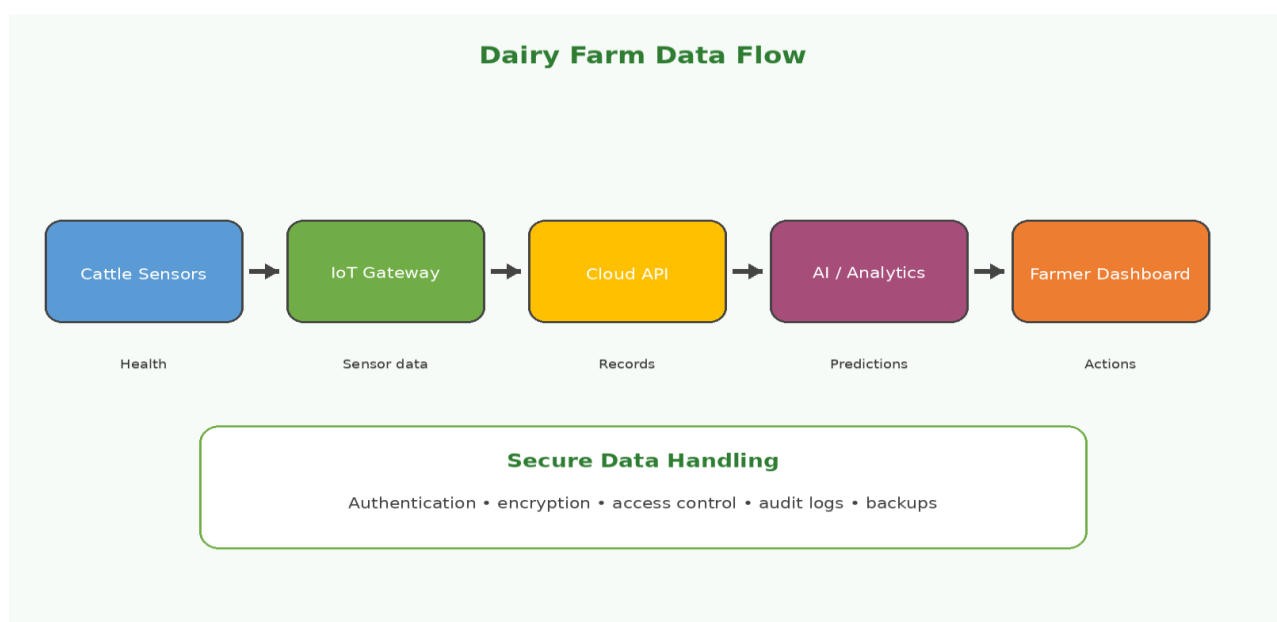


Figure 3. Smart Dairy Farm data flow from sensors to farmer actions

9. AI-Based Health Prediction and Monitoring

9.1 Health Monitoring

Continuous monitoring helps the platform identify unusual changes in cattle-related readings. The system combines sensor observations and stored records to calculate a health status or risk indicator. The purpose of the CI/CD pipeline is to ensure that the software implementing these calculations is tested before deployment.

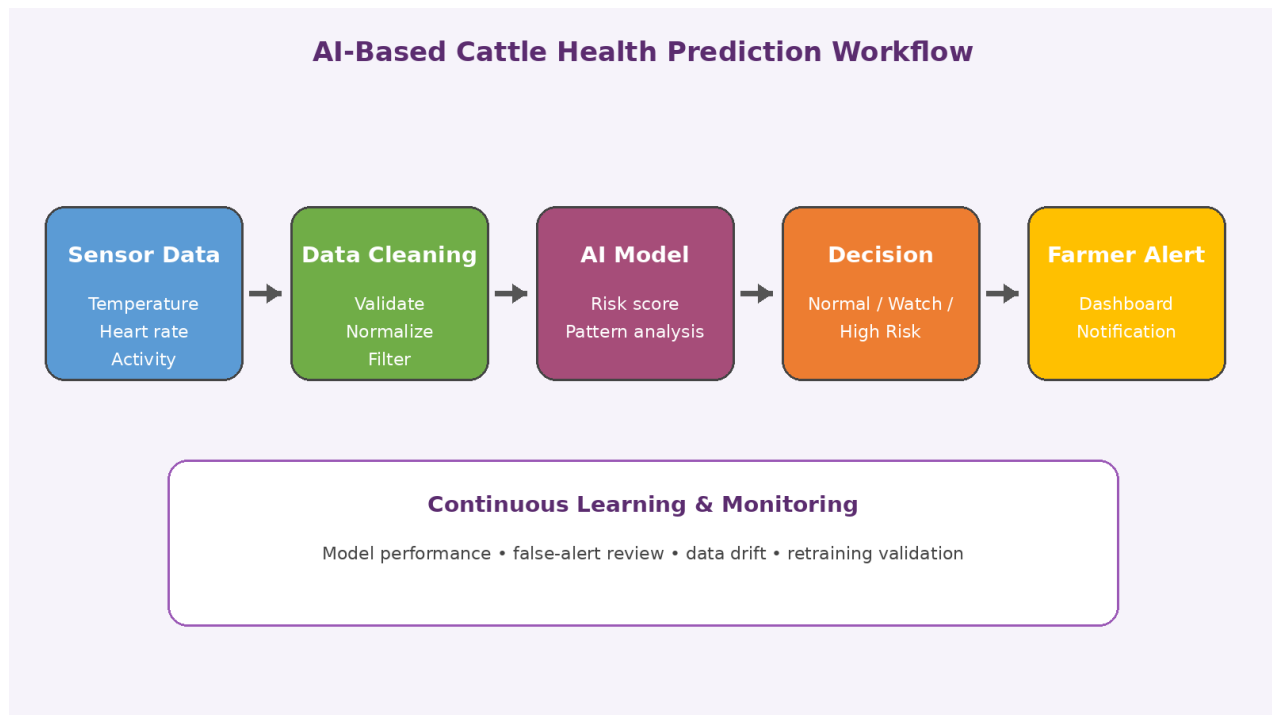


Figure 4. AI-based cattle health prediction workflow

9.2 AI Validation in CI/CD

- Validate the input data schema before model execution.
- Run automated tests on preprocessing and feature-generation logic.
- Use a controlled validation dataset during pipeline testing.
- Compare important model-quality measures against an accepted baseline.
- Check that model files and configuration versions are traceable.
- Do not automatically promote a model when critical validation checks fail.
- Monitor production predictions and investigate unexpected changes.

10. Farm Dashboard, Analytics and Operational Monitoring

10.1 Dashboard Functions

The dashboard provides a central view of farm information. It displays cattle health status, milk production, feeding efficiency, environmental information, and active alerts. The dashboard is tested as part of the release process because an unavailable or incorrect dashboard reduces the usefulness of the complete platform.

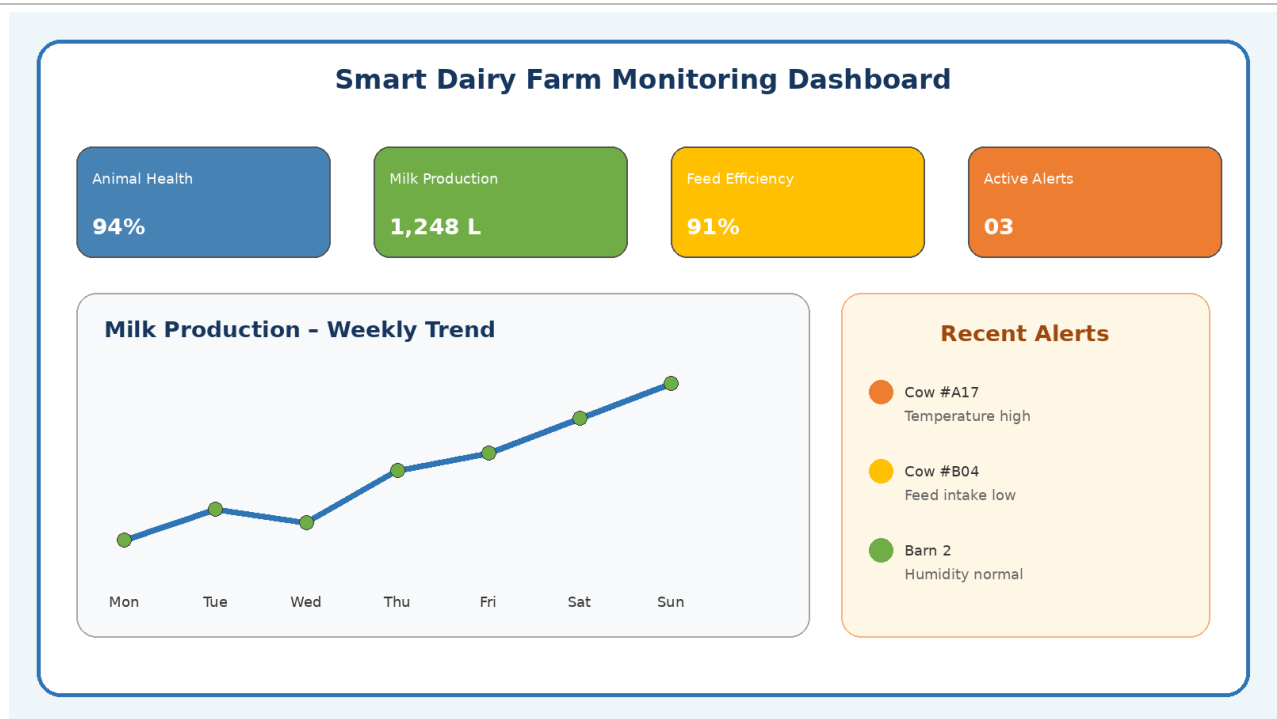


Figure 5. Colour smart dairy farm monitoring dashboard concept

10.2 Monitoring Metrics

- Application availability and response time.
- API error rate and failed requests.
- Sensor-ingestion success rate.
- Database and storage health.
- CPU, memory, and container health.
- Number of active alerts and failed processing jobs.
- Deployment success or failure status.

10.3 Notifications

The system notifies the responsible team when important pipeline or production events occur, including failed builds, failed tests, security findings, unsuccessful staging deployments, production deployment failures, and post-release health problems. Notifications are concise and actionable, identifying the failed stage, release version, and recommended next investigation step.

11. Sample CI/CD Pipeline Configuration

The extract below illustrates, in simplified pseudocode form, how the designed pipeline can be expressed as a GitHub Actions workflow. It automates build, test, quality and security scanning, containerisation, staging deployment, smoke testing, and a guarded production deployment with an automatic rollback trigger.

```
name: smart-dairy-ci-cd
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run unit & integration tests
        run: pytest --maxfail=1 --cov=app
      - name: Static code analysis (SonarQube)
        run: sonar-scanner
      - name: Dependency & container scan (Trivy)
        run: trivy fs --exit-code 1 .

  package-and-deploy-staging:
    needs: build-and-test
    runs-on: ubuntu-latest
    steps:
      - name: Build Docker image
        run: docker build -t dairy-app:${{ github.sha }} .
      - name: Push to registry
        run: docker push registry/dairy-app:${{ github.sha }}
      - name: Deploy to staging
        run: ./deploy.sh staging ${{{ github.sha }}}
      - name: Run smoke tests on staging
        run: pytest tests/smoke --env=staging

  deploy-production:
    needs: package-and-deploy-staging
    if: github.ref == 'refs/heads/main'
    environment: production
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to production
        run: ./deploy.sh production ${{ github.sha }}
      - name: Post-deployment health check
        run: ./health_check.sh production || ./rollback.sh production
```

This configuration demonstrates the direct link between the architecture described in Section 2 and an executable pipeline: every stage in the diagram (build, test, quality/security, package, staging, production) corresponds to a job or step in the workflow file, keeping design and implementation traceable to one another.

12. Team Roles and Responsibilities

A CI/CD pipeline is only as reliable as the people and processes around it. The table below assigns clear responsibility for each stage of the pipeline so that failures are investigated quickly and releases remain accountable.

Role	Responsibility in the CI/CD Process
Developer	Writes code, creates pull requests, fixes failed builds/tests, and responds to code-review comments.
Reviewer / Tech Lead	Reviews pull requests, approves merges to protected branches, ensures design and coding standards are followed.
DevOps / Release Engineer	Maintains the CI/CD pipeline configuration, manages the container registry, and coordinates deployments.
QA / Test Engineer	Designs automated test cases, monitors test coverage, and investigates recurring test failures.
Security Engineer	Reviews security-scan findings, manages secrets, and enforces access-control policies.
Farm Operations Lead	Validates staging releases from an operational point of view before production approval.
System Administrator	Monitors production infrastructure, responds to alerts, and executes rollback when required.

13. Risk Management, Benefits and Expected Outcomes

13.1 Risks and Mitigation

Risk	Impact	Mitigation
Faulty release	Service interruption	Automated tests, staging, health checks, rollback
Security vulnerability	Data or service exposure	Dependency, code, secret and container scans
Bad sensor data	Incorrect analytics	Validation rules and anomaly checks
AI performance degradation	Poor predictions	Model validation and monitoring
Configuration mismatch	Deployment failure	Containers and environment-specific configuration
Unauthorized change	Operational risk	Protected branches and approvals

13.2 Expected Benefits

- Improved livestock health monitoring.
- Increased visibility of milk production.
- Reduced risk of software defects reaching production.
- More consistent and repeatable software releases.
- Improved security and traceability.
- Faster detection and recovery from deployment failures.
- Better support for data-driven farm management.
- Reduced manual effort in software delivery and operational monitoring.

13.3 Expected Outcomes

The completed system is expected to support improved livestock health, increased milk production, reduced disease outbreaks, better farm management, lower operational costs, enhanced animal welfare, data-driven farming decisions, and increased farmer profitability. The CI/CD pipeline supports these outcomes indirectly by keeping the software platform reliable, testable, secure, and maintainable.

14. Implementation Plan

- Create the Git repository and establish branch protection.
- Organize the application into testable services or modules.
- Create the CI workflow for build and automated tests.
- Add code-quality and dependency/security scanning.
- Create Docker images and publish versioned artifacts.
- Create a staging environment and automated deployment workflow.
- Add production approval gates and health checks.
- Configure monitoring, notifications, and rollback.
- Review pipeline results and improve failed or slow stages.

15. Conclusion

The Intelligent AI-Based Smart Dairy Farm Management System requires a dependable software delivery process because it brings together livestock monitoring, IoT data, AI prediction, feeding optimization, milk-production tracking, cloud services, and real-time analytics. The CI/CD pipeline proposed in this document provides a systematic way to move software changes from development to production while applying automated testing, quality analysis, security validation, packaging, staging, deployment, monitoring, and rollback. This reduces dependence on manual release activities and provides clear evidence that each version has passed the required checks.

The proposed design also supports continuous improvement. Developers receive rapid feedback from failed tests and quality gates, while operations teams can monitor production behaviour and respond to alerts. Versioned artifacts and rollback mechanisms improve recovery when a release does not behave as expected. For the Smart Dairy Farm system, this approach helps protect the availability and reliability of services used to monitor cattle, track milk production, optimize feeding, and provide farm analytics.

Overall, the CI/CD design satisfies the assessment focus on requirement analysis, pipeline architecture, tool selection, automated testing, quality assurance, deployment, security, rollback, diagrams, and documentation. The combination of a structured DevOps workflow and the proposed smart-dairy architecture provides a practical foundation for delivering reliable software that can support modern, data-driven dairy farm management.